深度学习详解（Datawhale、李宏毅）——第 2 章 实践方法论

在应用机器学习算法时，实践方法论能够帮助我们更好地训练模型。如果在 Kaggle 上的结果不太好，虽然 Kaggle 上呈现的是测试数据的结果，但要先检查训练数据的损失。看看模型在训练数据上面，有没有学起来，再去看测试的结果，如果训练数据的损失很大，显然它在训练集上面也没有训练好。接下来再分析一下在训练集上面没有学好的原因。

2.1 模型偏差

模型偏差可能会影响模型训练。当所选函数过于简单时，该函数的集合太小了，可以让损失变低的函数不在模型可以描述的范围内。在这种情况下，就算找出了一个 θ^*，虽然它是这些蓝色的函数里面最好的一个，但损失还是不够低。这种情况就是想要在大海里面捞针（一个损失低的函数），结果针根本就不在海里。

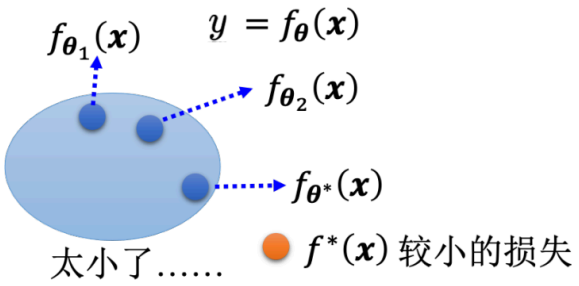


图 2.1　模型太简单的问题

这个时候重新设计一个模型，给模型更大的灵活性。其中一个做法是增加输入的特征；另一个做法是利用深度学习，提升网络的灵活性。即便如此，也不意味着训练的时候损失大就一定要归咎于模型偏差，还可能是因为优化做得不好。

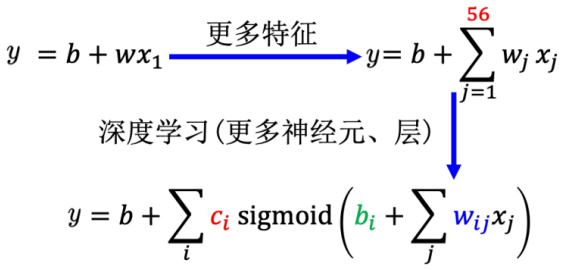
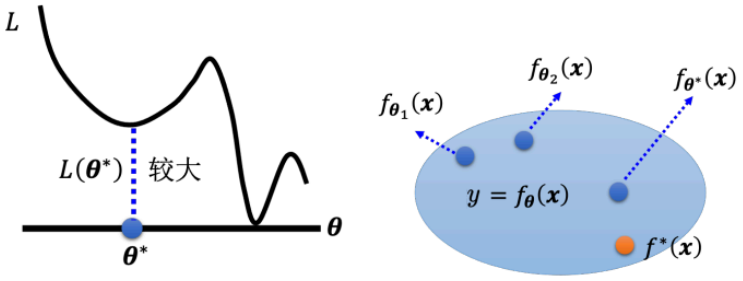


图 2.2　增加模型的灵活性

2.2 优化问题

一般只会用到梯度下降进行优化，这种优化的方法很多的问题。比如可能会卡在局部最小值的地方，无法找到一个真的可以让损失很低的参数。梯度下降是解一个优化的问题，找到 θ^* 就结束了。但 θ^* 的损失不够低。此时，到底是模型偏差还是优化的问题呢？找不到一个损失低的函数，到底是因为模型的灵活性不够，还是因为模型的灵活性已经够了，知识梯度下降过程没有达到预期？

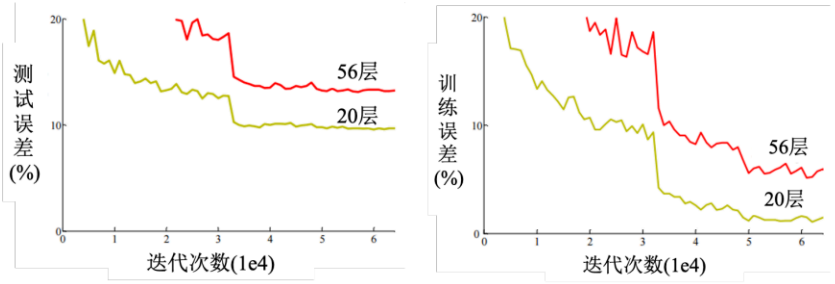


(a) 优化过程中会卡在局部最小值　　(b) 模型的灵活性足够大

图 2.3　优化方法的问题

一个建议判断的方法，通过比较不同的模型来判断模型现在到底够不够大。举个例子，这一个实验是从残差网络的论文“Deep Residual Learning for Image Recognition” 里面节录出来的。这篇论文在测试集上测试两个网络，一个网络有 20 层，一个网络有 56 层。由图可知，这代表 56 层的网络的优化没有做好，它的优化不给力。

> 注意，这不是过拟合，因为 56 层的网络在训练集和测试集上都不好。



(a) 测试数据　　(b) 训练数据

图 2.4　残差网络的例子

> Q：如何知道是 56 层的优化不给力，搞不好是模型偏差，搞不好是 56 层的网络的模型灵活性还不够大，它要 156 层才好，56 层也许灵活性还不够大？
> A：但是比较 56 层跟 20 层，20 层的损失都已经可以做到这样了，56 层的灵活性一定比 20 层更大。如果 56 层的网络要做到 20 层的网络可以做到的事情，对它来说是轻而易举的。它只要前 20 层的参数，跟这个 20 层的网络一样，剩下 36 层就

> 什么事都不做，复制前一层的输出就好了。如果优化成功，56 层的网络应该要比 20 层的网络可以得到更低的损失。但结果在训练集上面没有，这个不是过拟合，这个也不是模型偏差，因为 56 层网络灵活性是够的，这个问题是优化不给力，优化做得不够好。

这边给大家的建议是看到一个从来没有做过的问题，可以先跑一些比较小的、比较浅的网络，或甚至用一些非深度学习的方法，比如线性模型、**支持向量机（Support Vector Machine，SVM）**，SVM 可能是比较容易做优化的，它们比较不会有优化失败的问题。也就是这些模型它会竭尽全力的，在它们的能力范围之内，找出一组最好的参数，它们比较不会有失败的问题。因此可以先训练一些比较浅的模型，或者是一些比较简单的模型，先知道这些简单的模型，到底可以得到什么样的损失。

接下来还缺一个深的模型，如果深的模型跟浅的模型比起来，深的模型明明灵活性比较大，但损失却没有办法比浅的模型压得更低代表说优化有问题，梯度下降不给力，因此要有一些其它的方法来更好地进行优化。

如果训练数据上面的损失小，测试数据上的损失大，可能是真的过拟合。在测试上的结
果不好，不一定是过拟合。要把训练数据损失记下来，先确定优化没有问题，模型够大了。接下来才看看是不是测试的问题，如果是训练损失小，测试损失大，这个有可能是过拟合。

2.3 过拟合

模型的灵活性越高，其拟合程度越大，但是也会有可能导致拟合过大，产生奇怪的曲线，导致过拟合。

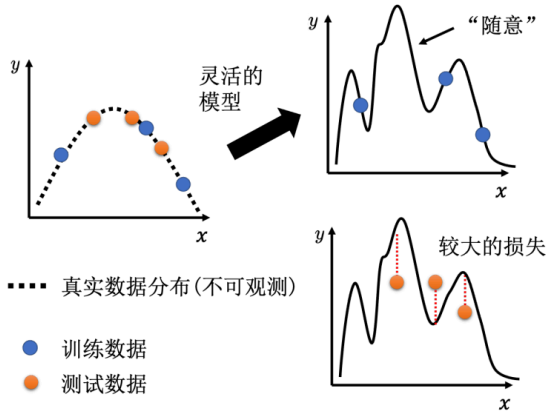


图 2.6　模型灵活导致的问题

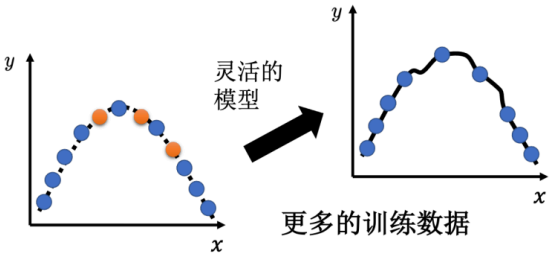


图 2.7　增加数据

怎么解决过拟合的问题呢？有两个可能的方向：

第一个方向是往往是最有效的方向，即增加训练集。如图 2.7 所示。可以做**数据增强（data augmentation）**，这个方法并不算是使用了额外的数据。

数据增强就是根据问题的理解创造出新的数据。举个例子，在做图像识别的时候，常做的一个招式是，假设训练集里面有某一张图片，把它左右翻转，或者是把它其中一块截出来放大等等。对图片进行左右翻转，数据就变成两倍。但是数据增强不能够随便乱做。在图像识别里面，很少看到有人把图像上下颠倒当作增强。因为这些图片都是合理的图片，左右翻转图片，并不会影响到里面的内容。但把图像上下颠倒，可能不是一个训练集或真实世界里面会出现的图像。如果给机器根据奇怪的图像学习，它可能就会学到奇怪的东西。所以数据增强，要根据对数据的特性以及要处理的问题的理解，来选择合适的数据增强的方式。

另外一个解法是给模型一些限制，让模型不要有过大的灵活性。

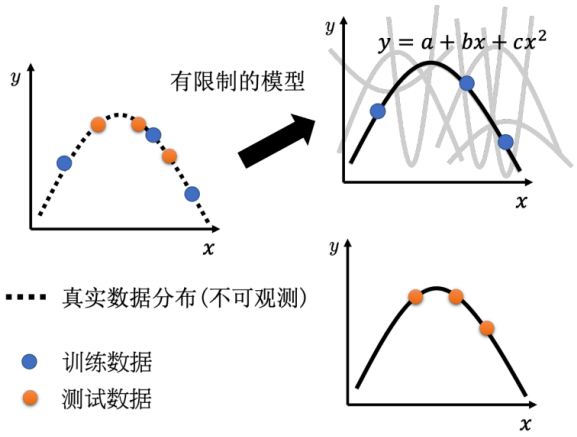


图 2.8　对模型增加限制

解决过拟合的问题，要给模型一些限制，具体来说，有如下方法：

- 给模型比较少的参数。如果是深度学习的话，就给它比较少的神经元的数量，本来每层一千个神经元，改成一百个神经元之类的，或者让模型共用参数，可以让一些参数有一样的数值。**全连接网络（fully-connected network）**其实是一个比较有灵活性的架构，而**卷积神经网络（Convolutional Neural Network，CNN）**是一个比较有限制的架构。
- 用比较少的特征。
- 还有别的方法，比如**早停（early stopping）**、**正则化（regularization）**和**丢弃法（dropout method）**。

即便如此，也不要给模型太多的限制。

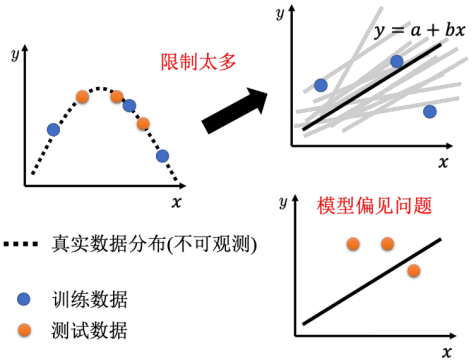


图 2.9　限制太大会导致模型偏差

模型的复杂程度和灵活性没有明确的定义，比较复杂的模型包含的函数比较多，参数也比较多。如图 2.10 所示，随着模型越来越复杂，训练损失可以越来越低，但测试时，当模型越来越复杂的时候，刚开始，测试损失会跟著下降，但是当复杂的程度，超过某一个程度以后，测试损失就会突然暴增了。这就是因为当模型越来越复杂的时候，复杂到某一个程度，过拟合的情况就会出现，所以在训练损失上面可以得到比较好的结果。在测试损失上面，会得到比较大的损失，可以选一个中庸的模型，不是太复杂的，也不是太简单的，刚刚好可以在训练集上损失最低，测试损失最低。

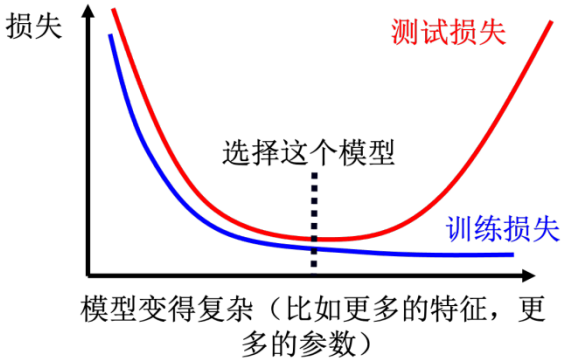


图 2.10　模型的复杂程度与损失的关系

Kaggle 中测试集分成公开的数据集跟私人的数据集，公开的分数可以看到，私人的分数要截止日期以后才知道。如果根据公开数据集来选择模型，可能会出现这种情况：在公开的排行榜上面排前十，但是截止日期一结束，可能掉到 300 名之外。因为计算分数的时候，会同时考虑公开和私人的分数。随机输出多个结果，暴力多次提交，总有一次得分非常的高。

> Q：为什么要把测试集分成公开和私人？
> A：假设所有的数据都是公开，就算是一个一无是处的模型，它也有可能在公开的数据上面得到好的结果。如果只有公开的测试集，没有私人测试集，写一个程序不断随机产生输出就好，不断把随机的输出上传到 Kaggle，可以随机出一个好的结果。这个显然没有意义。而且如果公开的测试数据是公开的，公开的测试数据的结

> 果是已知的，一个很废的模型也可能得到非常好的结果。不要用公开的测试集调模型，因为可能会在私人测试集上面得到很差的结果，不过因为在公开测试集上面的好的结果也有算分数。

2.4 交叉验证

比较合理选择模型的方法是把**训练的数据**分成两半，一部分称为**训练集（training set）**，一部分是**验证集（validation set）**。在训练集上训练出来的模型会使用验证集来衡量它们的分数，根据验证集上面的分数去挑选结果，再把这个结果上传到 Kaggle 上面得到的公开分数。在挑分数的时候，是用验证集来挑模型，所以公开测试集分数就可以反映私人测试集的分数。但假设这个循环做太多次，根据公开测试集上的结果调整模型太多次，就又有可能在公开测试集上面过拟合，在私人测试集上面得到差的结果。不过上传的次数有限制，所以无法走太多次循环，可以避免在公开的测试集上面的结果过拟合。根据过去的经验，就在公开排行榜上排前几名的，往往私人测试集很容易就不好。

其实最正确、最科学的做法是：只看**验证集损失**，哪个模型验证损失最小，就选哪个。完全不要去看公开测试集（public test）的分数，更不要根据它改模型。但现实中很难做到这么理想，因为人总会忍不住看公开测试集的结果，然后去微调模型。理想状态就是：只用验证集选模型；等你跑出一个还不错的基线模型（一个基础可用的模型）后，就别再乱调了，这样才能避免模型在测试集上过拟合。

但是这边会有一个问题，如果随机分验证集，可能会分得不好，分到很奇怪的验证集，会导致结果很差，如果有这个担心的话，可以用 k **折交叉验证（k-fold cross validation）**，如图 2.11 所示。

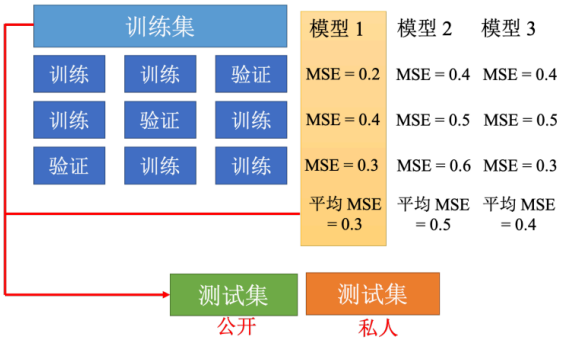


图 2.11　k 折交叉验证

k 折交叉验证就是先把训练集切成 k 等份。在这个例子，训练集被切成 3 等份，切完以后，拿其中一份当作验证集，另外两份当训练集，这件事情要重复 3 次。

2.5 不匹配

无论什么模型，对某一情况的预测都与真实值相差较大（模型根本没见过某一种输出模式），那么就会出现另外一种错误的形式，这种错误的形式称为**不匹配（mismatch）**。

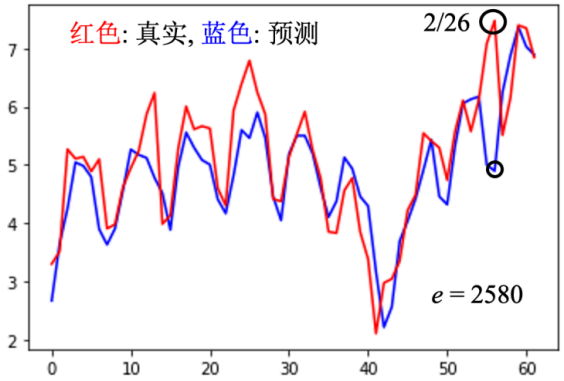


图 2.13　另一种错误形式：不匹配

不匹配跟过拟合其实不同，一般的过拟合可以用搜集更多的数据来克服，但是不匹配是
指训练集跟测试集的分布不同，训练集再增加其实也没有帮助了。图 2.14 是图像分类中的不匹配问题。增加数据也不能让模型做得更好，所以这种问题要怎么解决，匹不匹配要看对数据本身的理解了，我们可能要对训练集跟测试集的产生方式有一些理解，才能判断它是不是遇到了不匹配的情况。

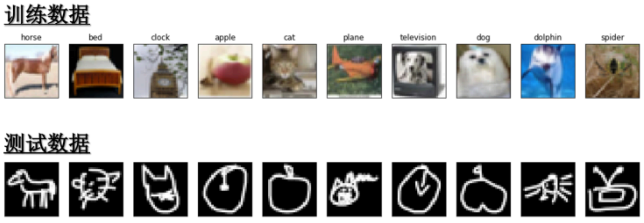


图 2.14　图像分类的不匹配问题